INTEGRAÇÃO DO LLVM E ICARUS VERILOG PARA COMPILAÇÃO, SIMULAÇÃO E ESTUDO DE PROCESSADORES PERSONALIZADOS

Murillo Inacio da Costa Silva [1]
Icaro Mayan Lima Pereira [2]
Danilo Lacerda Silva [3]

RESUMO

Este artigo apresenta uma metodologia para a integração do LLVM e do Icarus Verilog, com o objetivo de realizar a compilação e simulação de processadores customizados. O estudo inclui uma breve introdução ao contexto e à relevância do tema, descreve os objetivos do trabalho e conclui com os principais resultados obtidos. Nosso objetivo é fornecer uma abordagem prática e acessível para desenvolvedores e pesquisadores que desejam explorar essa combinação de ferramentas, considerando a escassez de fontes didáticas sobre o assunto.

Palavras-chave: Compilação. LLVM. Processadores personalizados. Simulação.

ABSTRACT

This paper presents a methodology for the integration of LLVM and Icarus Verilog, aiming at the compilation and simulation of custom processors. The study includes a brief introduction to the context and importance of the topic, describes the objectives of the work, and concludes with the main findings. Our goal is to provide a practical and accessible approach for developers and researchers who wish to explore this combination of tools, given the scarcity of didactic sources on the subject

Keyworks: Compilation. LLVM. Custom Processors. Simulation.

INTRODUÇÃO

O avanço das arquiteturas de processadores tem sido um dos principais motores do progresso tecnológico nas últimas décadas. Com o aumento da complexidade dos sistemas computacionais, cresce também a necessidade de ferramentas que permitam projetar, compilar e simular processadores customizados de forma eficiente.

[1] Graduado pelo Curso de Ciências da computação da Universidade Federal do Cariri (UFCA) - CE, murillo.inacio@aluno.ufca.edu.br;
[2] Graduado pelo Curso de Ciências da computação da Universidade Federal do Cariri (UFCA) - CE, icaro.mayan@aluno.ufca.edu.br;
[3] Graduando pelo Curso de Ciências da computação da Universidade Federal do Cariri (UFCA) - CE, danilo.lacerda@aluno.ufca.edu.br;

Nesse contexto, o LLVM (Low-Level Virtual Machine) e o Icarus Verilog se destacam como tecnologias complementares. O LLVM fornece uma infraestrutura robusta e modular para compilação, otimização e geração de código em múltiplas arquiteturas, enquanto o Icarus Verilog é amplamente utilizado para modelagem e simulação de circuitos digitais descritos em Verilog. A integração dessas ferramentas pode resultar em um fluxo de trabalho que abrange desde a descrição do hardware até a geração de código executável, acelerando a validação e o desenvolvimento de novas arquiteturas.

REFERENCIAL TEÓRICO

Low-Level Virtual Machine (LLVM)

O **LLVM** é um conjunto de ferramentas de compilação que fornece uma infraestrutura sólida para otimização e geração de código. Desenvolvido inicialmente pela **Universidade de Illinois**, o LLVM foi projetado para ser modular e reutilizável, facilitando a integração com diversas linguagens de programação e plataformas de hardware. O LLVM é composto por três componentes principais: **front-end**, **middle-end** e **back-end**. O *front-end* traduz o código-fonte para uma representação intermediária (**IR**), o *middle-end* realiza otimizações nessa representação e o *back-end* gera o código de máquina específico para a arquitetura alvo (Project, 2021).

Icarus Verilog

O **Icarus Verilog** é uma implementação open-source do compilador **Verilog**, projetada para síntese e simulação de projetos digitais. Por ser leve e eficiente, é amplamente utilizado para validar arquiteturas em diferentes níveis de abstração, permitindo detectar falhas e otimizar projetos antes da implementação física (Williams 2019).

Integração LLVM e Icarus Verilog

A combinação das duas ferramentas possibilita um ciclo de desenvolvimento integrado: enquanto o LLVM cuida da compilação e otimização de programas, o Icarus Verilog simula o hardware, permitindo que arquiteturas customizadas sejam testadas de ponta a ponta

Documentação usada para integrar Cpu0 e LLVM

Durante o desenvolvimento deste trabalho, foi realizado uma extensa pesquisa para habilitar o LLVM para reconhecer o Cpu0 como um alvo válido, as principais fontes usadas incluem:

- A documentação oficial do LLVM (Project 2024);
- Source Code Artisan: Setting Up a New Backend (Artisan 2020);
- Repositórios dos usuários Jonathan2251 (Jonathan2251 2024) e do P2Tree (P2Tree 2024) no GitHub.

METODOLOGIA

A metodologia adotada consistiu em preparar o LLVM para reconhecer o **Cpu0** como arquitetura alvo e, em seguida, validar a integração com simulações em Verilog. As principais etapas foram:

Configuração do LLVM com suporte ao Cpu0:

O objetivo inicial do projeto foi compilar uma versão personalizada do LLVM com suporte ao alvo Cpu0, utilizando o repositório lbd (Jonathan2251 2024), integrado à versão 12.x do LLVM. Esta seção descreve as etapas realizadas para alcançar esse objetivo.

Processo de Build do LLVM

O processo de build do LLVM foi realizado em um diretório separado para organizar os arquivos gerados durante a configuração e compilação. A configuração inicial foi feita com o cmake, utilizando parâmetros específicos para habilitar o alvo Cpu0 e otimizar a geração do tableGen.

Geração de código Assembly para a arquitetura do Cpu0

Para testar a integração entre o LLVM e a arquitetura Cpu0, basta seguir uma sequência de etapas para gerar o código assembly a partir de um programa em C++ (neste caso, chamado **main_run.cpp**). Inicialmente, o programa main run.cpp foi compilado especificando o clang para o alvo MIPS, gerando um arquivo de bytecode, em seguida, o arquivo **main_run.bc** é convertido em **main_run.cpu0.o** utilizando os parâmetros **-march=cpu0**, que instrui o **llc** a usar o backend Cpu0 para gerar o arquivo objeto, e **-relocation-model=pic**, que define o modelo de realocação escolhido como **pic** (*Position-Independent Code*). Isso permite que o código gerado seja independente de endereços absolutos, tornando possível carregar o código em qualquer endereço de memória sem necessidade de recompilação ou ajustes no código.

A etapa final consiste em *desassemblar* o objeto binário gerado para obter uma representação em assembly legível, utilizando o **LLVM-objdump**, uma ferramenta que faz parte do projeto LLVM e é usada para examinar o conteúdo de arquivos binários. Esse processo gera um arquivo *.hex* que pode ser lido pela arquitetura do Cpu0 usando Verilog.

Figura 1: Exemplo de código gerado em Hexadecimal

```
/* main:*/
/*0:*/ 09 dd ff f0      /*addiu $sp, $sp, -16*/
/*4:*/ 02 cd 00 0c      /*st    $fp, 12($sp)*/
/*8:*/ 11 cd 00 00      /*move  $fp, $sp*/
/*c:*/ 09 20 00 00      /*addiu $2, $zero, 0*/
/*10:*/ 02 2c 00 08     /*st    $2, 8($fp)*/
/*14:*/ 02 2c 00 04     /*st    $2, 4($fp)*/
/*18:*/ 01 2c 00 04     /*ld    $2, 4($fp)*/
/*1c:*/ 09 22 00 01     /*addiu $2, $2, 1*/
/*20:*/ 02 2c 00 04     /*st    $2, 4($fp)*/
/*24:*/ 01 2c 00 04     /*ld    $2, 4($fp)*/
/*28:*/ 09 22 00 01     /*addiu $2, $2, 1*/
/*2c:*/ 02 2c 00 04     /*st    $2, 4($fp)*/
/*30:*/ 01 2c 00 04     /*ld    $2, 4($fp)*/
/*34:*/ 11 dc 00 00     /*move  $sp, $fp*/
/*38:*/ 01 cd 00 0c     /*ld    $fp, 12($sp)*/
/*3c:*/ 09 dd 00 10     /*addiu $sp, $sp, 16*/
/*40:*/ 3c e0 00 00     /*ret   $lr*/
/*44:*/ 00 00 00 00     /*nop*/
```

Fonte: Do próprio autor

Validação do circuito do Cpu0 em Verilog

Para verificar as modificações internas diretamente no circuito Cpu0, utilizamos simulações para registrar e analisar os sinais internos do processador. Inicialmente, adicionamos as seguintes linhas ao módulo **"main"** no arquivo **Cpu0.v**. A primeira linha inserida gera um arquivo *.vcd*, contendo alguns dos componentes do circuito a serem examinados durante a execução dos testes. A segunda linha, por sua vez, indica qual escopo será analisado.

Figura 2: Modificações no arquivo Cpu0.v

```
$dumpfile("test.vcd");
$dumpvars(0, main);
```

Fonte: Do próprio autor

Em seguida, para gerar e executar o arquivo .vvp, que executará o teste do arquivo cpu0.v contendo os circuitos do Cpu0, basta executar os seguintes comandos:

Figura 3: Comando para gerar o arquivo *.vpp*

```
iverilog -o test.vvp cpu0.v
```

Fonte: Do próprio autor

Figura 4: Comando para executar o arquivo *.vpp*

```
vvp test.vvp
```

Fonte: Do próprio autor

Modificando a arquitetura do Cpu0

Uma das instruções na arquitetura Cpu0 foi ocultada, tornando-a "invisível" para o LLVM. Isso foi feito modificando os arquivos responsáveis por descrever o conjunto de instruções no backend Cpu0. Inicialmente, simplesmente omitimos a definição da instrução no arquivo **Cpu0InstrInfo.td**, entretanto, essa modificação isolada resulta em um erro, indicando que o LLVM não conseguiu encontrar uma implementação válida para a instrução removida durante a etapa de seleção de instruções. Para resolver o erro e concluir a remoção da instrução, foi utilizado o comando **setOperationAction** no arquivo **Cpu0ISelLowering.cpp**, especificamente dentro do construtor da classe **Cpu0TargetLowering**. Essa função permite definir como o compilador deve tratar determinadas operações para uma arquitetura alvo, sendo útil para modificar o comportamento padrão do compilador em operações específicas.

Figura 5: Remoção da definição da instrução no arquivo Cpu0InstrInfo.td

```
def MUL: ArithLogicR<0x17, "mul", mul, IIImul, CPURegs,
   1>;
```

Fonte: Do próprio autor

Figura 6: Adicionando comandos para expandir a instrução MUL em uma sequência de instruções equivalentes

```
setOperationAction(ISD::MUL, MVT::i32, Expand);
setOperationAction(ISD::MULHS, MVT::i32, Expand);
setOperationAction(ISD::MULHU, MVT::i32, Expand);
```

Fonte: Do próprio autor

Com isso, para observar os impactos da remoção da instrução **MUL**, testamos um trecho de código e analisamos o código hexadecimal gerado antes e depois das modificações.

Figura 7: Função usada para testar os impactos de remover uma instrução no código hexadecimal gerado

```
int main() {
  int a = 1;
  int b = 2;
  a = a * b;
  b = b * 3;
  return 0;
}
```

Fonte: Do próprio autor

Figura 8: Comparação lado a lado do código hexadecimal gerado antes de depois da remoção do MUL

```
/*0: * /        09 dd ff f0     /*addiu $sp, $sp, -16 */
/*4: * /        02 cd 00 0c     /*st    $fp, 12($sp)*/
/*8: * /        11 cd 00 00     /*move  $fp, $sp*/
/*c: * /        09 20 00 00     /*addiu $2, $zero, 0*/
/*10: */        02 2c 00 08     /*st    $2, 8($fp)*/
/*14: */        09 30 00 01     /*addiu $3, $zero, 1*/
/*18: */        02 3c 00 04     /*st    $3, 4($fp)*/
/*1c: */        09 30 00 02     /*addiu $3, $zero, 2*/
/*20: */        02 3c 00 00     /*st    $3, 0($fp)*/
/*24: */        01 3c 00 04     /*ld    $3, 4($fp)*/
/*28: */        01 4c 00 00     /*ld    $4, 0($fp)*/
/*2c: */        17 33 40 00     /*mul   $3, $3, $4*/
/*30: */        02 3c 00 04     /*st    $3, 4($fp)*/
/*34: */        01 3c 00 00     /*ld    $3, 0($fp)*/
/*38: */        09 40 00 03     /*addiu $4, $zero, 3*/
/*3c: */        17 33 40 00     /*mul   $3, $3, $4*/
/*40: */        02 3c 00 00     /*st    $3, 0($fp)*/
/*44: */        11 dc 00 00     /*move  $sp, $fp*/
/*48: */        01 cd 00 0c     /*ld    $fp, 12($sp)*/
/*4c: */        09 dd 00 10     /*addiu $sp, $sp, 16*/
/*50: */        3c e0 00 00     /*ret   $lr*/
/*54: */        00 00 00 00     /*nop*/
```

```
/* 0: */        09 dd ff f0     /*addiu $sp, $sp, -16*/
/* 4: */        02 cd 00 0c     /*st    $fp, 12($sp)*/
/* 8: */        11 cd 00 00     /*move  $fp, $sp*/
/* c: */        09 20 00 00     /*addiu $2, $zero, 0*/
/* 10: */       02 2c 00 08     /*st    $2, 8($fp)*/
/* 14: */       09 30 00 01     /*addiu $3, $zero, 1*/
/* 18: */       02 3c 00 04     /*st    $3, 4($fp)*/
/* 1c: */       09 30 00 02     /*addiu $3, $zero, 2*/
/* 20: */       02 3c 00 00     /*st    $3, 0($fp)*/
/* 24: */       01 3c 00 04     /*ld    $3, 4($fp)*/
/* 28: */       01 4c 00 00     /*ld    $4, 0($fp)*/
/* 2c: */       41 34 00 00     /*mult  $3, $4*/
/* 30: */       47 30 00 00     /*mflo  $3*/
/* 34: */       46 40 00 00     /*mfhi  $4*/
/* 38: */       02 3c 00 04     /*st    $3, 4($fp)*/
/* 3c: */       01 3c 00 00     /*ld    $3, 0($fp)*/
/* 40: */       09 40 00 03     /*addiu $4, $zero, 3*/
/* 44: */       41 34 00 00     /*mult  $3, $4*/
/* 48: */       47 30 00 00     /*mflo  $3*/
/* 4c: */       46 40 00 00     /*mfhi  $4*/
/* 50: */       02 3c 00 00     /*st    $3, 0($fp)*/
/* 54: */       11 dc 00 00     /*move  $sp, $fp*/
/* 58: */       01 cd 00 0c     /*ld    $fp, 12($sp)*/
/* 5c: */       09 dd 00 10     /*addiu $sp, $sp, 16*/
/* 60: */       3c e0 00 00     /*ret   $lr*/
/* 64: */       00 00 00 00     /*nop*/
```

Fonte: Do próprio autor

RESULTADOS

O LLVM pôde ser configurado com sucesso para reconhecer e compilar programas em linguagens como C++ para a arquitetura **Cpu0**. Os experimentos demonstraram que

- Foi possível gerar código executável em formato hexadecimal compatível com simulação em Verilog;

- As modificações na arquitetura foram refletidas tanto no compilador quanto na simulação, confirmando a flexibilidade da integração;
- O uso do **Icarus Verilog** em conjunto com o **LLVM** permitiu validar o comportamento do processador em diferentes cenários, fortalecendo a metodologia proposta.

CONSIDERAÇÕES FINAIS

Este trabalho demonstrou a viabilidade e as vantagens da integração do LLVM com o Icarus Verilog para o desenvolvimento e simulação de processadores customizados. O estudo apresentou uma metodologia prática para a compilação de código e a validação de arquiteturas de processadores por meio de etapas e exemplos detalhados, incluindo a adaptação bem-sucedida da arquitetura Cpu0. Os resultados destacam o potencial dessa integração para otimizar o fluxo de trabalho de pesquisadores e desenvolvedores, oferecendo tanto flexibilidade quanto precisão no desenvolvimento de processadores específicos para aplicações. Pesquisas futuras podem expandir essa base, explorando arquiteturas adicionais e refinando o processo para aumentar a usabilidade e a eficiência

REFERÊNCIAS

Artisan 2020 ARTISAN, S. C. How to Write an LLVM Backend #2: Setting Up a New Backend. 2020. Disponível em: ⟨https: //sourcecodeartisan.com/2020/09/20/llvm-backend-2.html⟩.

Jonathan2251 2024 JONATHAN2251. lbd: llvm backend document. 2024. Disponível em: ⟨https://github.com/Jonathan2251/lbd⟩.

Nery 2020 NERY, M. P. Estudo e desenvolvimento de uma ferramenta base para criação de processadores de aplicação específica. Revista de Computação, 2020.

P2Tree 2024 P2TREE. LLVM for cpu0: This is a tutorial to learn LLVM, I realize a backend to compiler machine code for cpu0 which is a simple RISC cpu. 2024. Disponível em: ⟨https://github.com/P2Tree/LLVM for cpu0/tree/main⟩.

Project 2021 PROJECT, L. LLVM Language Reference Manual. [S.l.], 2021. Disponível em: ⟨https://llvm.org/docs/LangRef.html⟩.

Williams 2019 WILLIAMS, S. Icarus Verilog. 2019. Disponível em: ⟨https://github.com/steveicarus/iverilog⟩.